

Learn Low-Level Design in 25 Days: Day 8

LLD Patterns Every Engineer Must Know: Singleton, Factory, Builder

TechPreneur

Published October 20, 2025

Updated October 21, 2025

Free: No

Read on Freedium · original

Contents

The Hidden Architecture Behind Everyday Code 0

1. The Singleton Pattern 0

Without Singleton 0

With Singleton 0

2. The Factory Pattern 0

Without Factory 0

With Factory 0

3. The Builder Pattern 0

With Builder 0

Visual Summary 0

Try It Yourself 0

Summary 0

The Hidden Architecture Behind Everyday Code

Ever wondered how a single instance of your database connection survives across hundreds of API calls without blowing up memory? Or how frameworks like Spring or Django magically create objects you never explicitly `new` up?

That's not luck — that's **design patterns** at work.

Welcome to **Day 8** of Learn Low-Level Design in 25 Days — our hands-on journey through real-world LLD. So far, we've talked about classes, objects, relationships, and how to turn ideas into systems. Today, we're stepping into the world of **patterns** — the reusable blueprints every seasoned engineer keeps in their toolkit.



By the end of this post, you'll understand three foundational creational patterns — **Singleton**, **Factory**, and **Builder** — and know when and why to use them in your own designs.

1. The Singleton Pattern

Goal: Ensure only one instance of a class exists and provide a global point of access to it.

Think of your **database connection** or **logging service** — you don't want multiple copies of these floating around. You just want one shared instance.

Without Singleton

```
DatabaseConnection db1 = new DatabaseConnection();
DatabaseConnection db2 = new DatabaseConnection(); // Oops, another one!

db1.connect();
db2.connect(); // Creates two separate connections
```

This wastes resources and risks conflicts.

With Singleton

```
public class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() { } // prevent direct instantiation

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public void connect() {
        System.out.println("Connected to DB");
    }
}
```

Now:

```
DatabaseConnection db1 = DatabaseConnection.getInstance();
DatabaseConnection db2 = DatabaseConnection.getInstance();

System.out.println(db1 == db2); // true
```

Real-world analogy: Think of the Singleton as a "CEO object." There can only be one CEO in a company — everyone refers to the same instance.

Use it when:

- You need exactly one instance (DB, cache, config, logger)
- You want controlled global access

Avoid it when:

- You need testability or parallelism — Singleton can lead to hidden dependencies if abused.

2. The Factory Pattern

Goal: Delegate object creation to a separate class, hiding the "how" from the client.

Imagine an app where you need to create multiple types of notifications — Email, SMS, Push. Without a factory, your code looks like a giant `if-else` jungle.

Without Factory

```
if (type.equals("EMAIL")) {
    return new EmailNotification();
} else if (type.equals("SMS")) {
    return new SMSNotification();
}
```

Every time you add a new type, you touch the same code. That's brittle.

With Factory

```
interface Notification {
    void notifyUser();
}

class EmailNotification implements Notification {
    public void notifyUser() { System.out.println("Email sent!"); }
}

class SMSNotification implements Notification {
    public void notifyUser() { System.out.println("SMS sent!"); }
}

class NotificationFactory {
    public static Notification createNotification(String type) {
        return switch (type) {
            case "EMAIL" -> new EmailNotification();
            case "SMS" -> new SMSNotification();
            default -> throw new IllegalArgumentException("Unknown type");
        };
    }
}
```

Usage:

```
Notification n = NotificationFactory.createNotification("EMAIL");
n.notifyUser(); // Email sent!
```

Analogy: A factory is like a restaurant kitchen. You don't know (or care) how your dish is cooked — you just order by name, and the kitchen handles it.

Use it when:

- Object creation logic is complex or variable
- You want to decouple creation from usage

Avoid it when:

- There's only one simple object type (unnecessary abstraction)

3. The Builder Pattern

Goal: Build complex objects step-by-step without needing a giant constructor.

Ever seen a constructor like this?

```
User user = new User("John", "Doe", "NY", "12345", true, false, "Male", 30);
```

Nobody remembers the order of those parameters. Builders fix this.

With Builder

```
public class User {
    private String firstName, lastName, city, zip;
    private boolean isActive, isVerified;
    private String gender;
    private int age;

    private User(Builder builder) {
        this.firstName = builder.firstName;
        this.lastName = builder.lastName;
        this.city = builder.city;
        this.zip = builder.zip;
        this.isActive = builder.isActive;
        this.isVerified = builder.isVerified;
        this.gender = builder.gender;
        this.age = builder.age;
    }

    public static class Builder {
        private String firstName, lastName, city, zip;
        private boolean isActive, isVerified;
        private String gender;
        private int age;

        public Builder firstName(String fn) { this.firstName = fn; return this; }
        public Builder lastName(String ln) { this.lastName = ln; return this; }
        public Builder city(String c) { this.city = c; return this; }
        public Builder age(int a) { this.age = a; return this; }

        public User build() { return new User(this); }
    }
}
```

Usage:

```
User user = new User.Builder()
                .firstName("John")
                .lastName("Doe")
                .city("NY")
                .age(30)
                .build();
```

Analogy: A builder is like a pizza order form — you pick your toppings one by one, and the chef assembles your perfect pizza.

Use it when:

- You have objects with many optional parameters

- You want readability and immutability

Avoid it when:

- Object construction is simple or rarely changes

Visual Summary

```
Singleton: One shared instance
 └── DatabaseConnection.getInstance()

Factory: Centralized object creator
 └── NotificationFactory.createNotification("SMS")

Builder: Step-by-step object construction
 └── new User.Builder().name("John").city("NY").build();
```

Try It Yourself

1. **Singleton Challenge:** Design a `Logger` class that ensures only one log instance is used across your application.
2. **Factory Challenge:** Build a `ShapeFactory` that returns objects like `Circle`, `Square`, or `Triangle` based on input.
3. **Builder Challenge:** Implement a `CarBuilder` that lets you configure a car's engine type, color, and transmission step-by-step.

Summary

- **Singleton** → One instance shared globally
- **Factory** → Hides complex object creation logic
- **Builder** → Simplifies building of complex objects

When to use these:

- Use **Singleton** for shared resources
- Use **Factory** when creation logic varies
- Use **Builder** for clean object construction

Tomorrow, we'll dive into **Adapter, Decorator, and Strategy Patterns** — and see how real-world systems like Uber and Spotify use them to stay flexible and scalable.

You can save the full series here: Learn Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)